StreamingApp — Technical File Reference

Docker · Docker Compose · Kubernetes · Helm · Terraform · Ansible · Jenkins

A field-by-field explanation of every configuration file created for this project, plus the commands run at each phase — from IAM user creation through AWS deployment.

Prepared for viva / oral-exam preparation.

Contents

1. Architecture Overview

The application is a microservices video-streaming platform: a React SPA **frontend** and four independent Node.js services — **auth-service** (3001), **streaming-service** (3002), **admin-service** (3003), **chat-service** (3004) — backed by a single MongoDB instance. Locally it runs via **docker-compose**. In production it runs on **AWS EKS**, provisioned by **Terraform**, configured by **Ansible** (for the Jenkins CI server), built/deployed by a **Jenkins** pipeline using **Helm**, exposed to the internet through an **AWS Load Balancer Controller**-managed ALB via two **Ingress** objects, with images stored in **ECR** and video files in **S3**.

2. Docker — Dockerfiles

frontend/Dockerfile

Multi-stage build: compiles the React app, then serves the static output with nginx.

INSTRUCTION	PURPOSE
`FROM node:18-alpine AS builder`	Stage 1 — small Alpine-based Node 18 image, named "builder" so stage 2 can copy from it.
`ARG REACT_APP_*`	Build-time variables (API URLs for auth/streaming/admin/chat) — must be ARG (not ENV) because Create-React-App bakes `REACT_APP_*` vars into the JS bundle at `npm run build` time, not at container runtime.
`ENV REACT_APP_*=$REACT_APP_*`	Promotes each ARG into an environment variable so the CRA build process (which reads `process.env`) can see it during `npm run build`.
`COPY package*.json ./ / RUN npm ci`	Installs dependencies before copying the rest of the source — a layer-caching optimization so `npm ci` only reruns when package.json changes, not on every source edit.
`RUN npm run build`	Produces the static production bundle in `/app/build`.
`FROM nginx:1.27-alpine`	Stage 2 — a fresh, much smaller final image; the Node toolchain from stage 1 is discarded.
`COPY --from=builder /app/build ...`	Pulls only the compiled static files from stage 1 into nginx's web root.
`COPY nginx.conf ...`	Custom nginx config (reverse-proxy/SPA routing rules).

`HEALTHCHECK ... http://127.0.0.1/health`	Uses the explicit IPv4 loopback, not `localhost` — `localhost` resolves to IPv6 `::1` inside the container where nginx doesn't listen (Issue #4 in the issues log).

backend/authService/Dockerfile

Single-stage Node image; simplest of the four backend Dockerfiles.

INSTRUCTION	PURPOSE
`FROM node:18-alpine AS base`	Base image for the whole build (no multi-stage split needed — no compile step).
`RUN npm install --production`	Skips devDependencies to keep the runtime image smaller.
`ENV NODE_ENV=production`	Signals to Express/other libraries to disable dev-only behaviors (verbose errors, etc.).
`EXPOSE 3001`	Documents the container's listening port (does not itself publish it — that's done at run/compose/k8s level).
`CMD ["npm", "run", "start"]`	Default process the container runs on start.

backend/adminService/Dockerfile

Adds a non-root user for defense-in-depth, and a HEALTHCHECK.

INSTRUCTION	PURPOSE
`COPY adminService/package*.json ./`	Path prefixed with the service folder name because the Docker build context for this file is the shared `backend/` directory, not `backend/adminService/` (fixed after Issue #1).
`addgroup/adduser -S`	`-S` creates a system group/user (no password, no home dir by default) — standard practice for a container process that should never need shell login.
`RUN chown -R appuser:appgroup /app`	Must run **before** `USER appuser` — ownership changes need root privileges; done in the wrong order this silently fails to fix the EACCES bug (Issue #2).
`USER appuser`	Drops root privileges for all subsequent instructions and for the running container process.
`HEALTHCHECK --start-period=15s`	Grace period before failed checks count against the container — avoids false-unhealthy during Node process startup.

backend/chatService/Dockerfile & backend/streamingService/Dockerfile

Same pattern as authService but with `WORKDIR /app/<serviceName>` and folder-prefixed COPY paths, since their build context is also the shared `backend/` directory.

3. Docker Compose

docker-compose.yml

Local multi-container orchestration for development — one Mongo + 4 backend services + frontend, wired together on the default bridge network.

FIELD	PURPOSE
`services.mongo.volumes`	`mongo-data:/data/db` — named volume so DB data survives container recreation.
`networks.default.aliases`	Registers an extra DNS name for each service on the compose network. Compose auto-registers only the service key (e.g. `auth`); the app code/nginx config expects `auth-service`, so an alias bridges the two (fixes Issue #3).
`environment: MONGO_URI`	Uses the compose-internal DNS name `mongo` — resolvable because Mongo and the backend services share the same Docker network.
`environment: ${VAR:-default}`	Shell-style default-value syntax — if `VAR` isn't set in the environment or an `.env` file, the value after `:-` is used.
`frontend.build.args`	Passes the `REACT_APP_*` build args through to the Dockerfile's `ARG` declarations at image-build time.
`frontend.ports "3000:80"`	Maps host port 3000 to the container's nginx port 80 — the only service exposed at a "normal" dev port; backend ports are exposed 1:1 for direct API testing.
`depends_on`	Controls container *start order* only — it does not wait for the app inside to be ready (that's what each service's own retry/health logic has to handle).

4. Kubernetes — Raw Manifests (k8s/)

These plain manifests were used for early local validation against a `kind` cluster before the project moved to the Helm-templated versions for the real AWS deployment. Concepts here carry over directly into the Helm templates.

k8s/namespace.yaml

FIELD	PURPOSE
`metadata.name: streamingapp`	All other manifests target this namespace — isolates the app's objects from other workloads in the cluster.
`labels.app.kubernetes.io/managed-by: helm`	Standard Helm convention label so Helm recognizes/tracks objects even when a namespace is pre-created outside a chart release.

k8s/secrets.yaml

FIELD	PURPOSE
`kind: Secret / type: Opaque`	Generic key-value secret (as opposed to typed secrets like `kubernetes.io/tls`).

`stringData (not data)`	Accepts plain-text values and lets Kubernetes base64-encode them automatically — `data:` would require the values to already be base64-encoded by hand.
`JWT_SECRET, MONGO_URI, AWS_*, etc.`	Consumed by every backend Deployment via `envFrom.secretRef` so all services share one secret object instead of five.

> Values are placeholders ("REPLACE_ME") in the committed file — real secrets are injected at apply time, never committed in plaintext.

k8s/mongo-statefulset.yaml

FIELD	PURPOSE
`kind: StatefulSet (not Deployment)`	Gives Mongo a stable network identity and stable storage across restarts — required for a stateful database, unlike the stateless app services.
`serviceName: mongo-service`	Must match a headless Service — required by StatefulSet for per-pod stable DNS.
`volumeClaimTemplates`	Unlike a Deployment's shared volume, this generates a unique PVC per pod replica automatically (here replicas=1, so one PVC: `mongo-data-mongo-0`).
`resources.requests / limits`	requests = what's reserved/guaranteed for scheduling; limits = hard ceiling before the container is throttled (CPU) or OOM-killed (memory).
`Service.clusterIP: None`	Makes the Service "headless" — no virtual IP/load-balancing; DNS resolves directly to pod IPs, which is what StatefulSet stable identity depends on.

k8s/auth-service.yaml, streaming-service.yaml, admin-service.yaml, chat-service.yaml

Each bundles a Deployment + Service (+ HPA where applicable) for one backend microservice. Structure is nearly identical across all four; differences noted below.

FIELD	PURPOSE
`spec.replicas`	Baseline pod count: 2 for auth/streaming/chat, 1 for admin (lower-traffic internal service).
`strategy.rollingUpdate: maxSurge 1 / maxUnavailable 0`	Zero-downtime deploys — always adds one extra new pod before removing an old one, never drops below the current replica count during a rollout.
`image: <AWS_ACCOUNT_ID>.dkr.ecr...`	Placeholder account ID — real manifests (and the Helm-templated equivalent) substitute the actual ECR registry at deploy time.
`envFrom.secretRef`	Bulk-imports every key from `streamingapp-secrets` as environment variables, rather than listing each one individually.
`livenessProbe vs readinessProbe`	Liveness failing → kubelet restarts the container. Readiness failing → pod is pulled out of the Service's load-balancing pool but left running. Both point at `/api/health` (except chat-service, which uses a raw `tcpSocket` check since it's a WebSocket-heavy service without a simple HTTP health endpoint on all paths).

`initialDelaySeconds / periodSeconds`	Liveness waits longer (20s) than readiness (10s) before the first check, giving the app time to boot before either probe can act — readiness checks more frequently since it only affects traffic routing, not restarts.
`HorizontalPodAutoscaler (auth/streaming/chat)`	Scales replicas by CPU utilization; streaming-service has the widest range (2–8) and lowest threshold (60%) since it's the most resource-intensive (video-related) service.
`Service.type: ClusterIP`	Internal-only virtual IP — not directly reachable from outside the cluster; external access goes through the Ingress/ALB instead.

k8s/frontend.yaml

FIELD	PURPOSE
`Service.metadata.name: frontend-service`	Named differently from the Deployment (`frontend`) in this raw manifest — **note:** the Helm chart instead names the Service identically to the Deployment (`frontend`), which is what the real Ingress backend must reference (see Issue #26 — a real bug from this exact naming mismatch).
`livenessProbe path: /health`	nginx-served static health endpoint, distinct from the backend services' `/api/health` .

k8s/ingress-frontend.yaml & k8s/ingress-api.yaml

FIELD	PURPOSE
`kubernetes.io/ingress.class: alb`	Tells the AWS Load Balancer Controller (not the default nginx ingress) to reconcile this object.
`alb.ingress.kubernetes.io/scheme: internet-facing`	Provisions a public ALB with a public DNS name, as opposed to `internal` (VPC-only).
`alb.ingress.kubernetes.io/target-type: ip`	Routes ALB traffic directly to pod IPs (via the VPC CNI) rather than to EC2 node ports — required for Fargate-style/ip-based routing and generally recommended over `instance` mode on EKS.
`alb.ingress.kubernetes.io/group.name: streamingapp`	The key fix from Issue #24 — both Ingress objects share this same group name so the controller provisions **one** ALB with rules merged from both objects, instead of one ALB each.
`alb.ingress.kubernetes.io/healthcheck-path`	Per-Ingress-object setting: `/health` on the frontend Ingress, `/api/health` on the API Ingress — this split is exactly why two Ingress objects were needed instead of one (a single Ingress only supports one healthcheck path globally).
`spec.rules[].http.paths[].path + pathType: Prefix`	API Ingress routes `/api/auth` , `/api/streaming` , `/api/admin` , and `/socket.io` (chat's WebSocket upgrade path) to their respective Services; frontend Ingress routes `/` (catch-all) to the frontend Service.

5. Helm Chart (helm/streamingapp/)

Chart.yaml

FIELD	PURPOSE
`apiVersion: v2`	Helm 3 chart format (v1 was Helm 2, unsupported).
`type: application`	Deployable chart (as opposed to `library`, which only provides reusable template snippets).
`version vs appVersion`	`version` is the chart's own release version; `appVersion` is just an informational label for which app version it packages — Helm doesn't enforce any relationship between the two.

values.yaml

FIELD	PURPOSE
`global.imageRegistry`	Shared prefix for every service's image reference — left as a placeholder in the committed file, overridden with `--set` at deploy time (see Issue #21) so the real AWS account ID never needs to be hardcoded in git.
`global.imageTag`	Applies uniformly to all services by default; the production values file overrides this per-deploy with the actual git commit SHA.
`services: (map, kebab-case keys)`	Each key (e.g. `auth-service`) becomes the literal Kubernetes object name when templates iterate over it — kebab-case chosen specifically to satisfy K8s RFC 1123 naming rules (Issue #6).
`services.<name>.enabled`	Lets a service be switched off entirely from a single flag — templates skip any service where this is false.
`services.<name>.hpa`	Present only on services that should autoscale (frontend and admin-service intentionally omit it — fixed replica counts).
`mongodb / ingress top-level keys`	Configuration for the non-templated parts of the deployment (Mongo storage size, Ingress host/cert ARN) — kept here for a single source of truth even though Mongo/Ingress aren't generated by the per-service range loop.

helm/values-prod.yaml

Production overlay applied on top of `values.yaml` via `-f helm/values-prod.yaml` — only overrides what differs from defaults (Helm merges the two).

FIELD	PURPOSE
`global.imageTag: "REPLACED_BY_JENKINS_AT_BUILD_TIME"`	Literal placeholder — the Jenkinsfile's Deploy stage always overrides this again with `--set global.imageTag=${IMAGE_TAG}` (the real commit SHA), so this value is never actually used as-is.
`authService/streamingService/chatService replicaCount`	Higher baseline replica counts for production than the chart defaults — reflects real traffic expectations vs. local/dev testing.

`streamingService.hpa.maxReplicas: 12`	Raises the autoscale ceiling for production load on the most resource-intensive service.

templates/deployment.yaml

SYNTAX	PURPOSE
`{{- range $svcName, $svc := .Values.services }}`	Loops over every key in the `services` map, binding the key to `$svcName` and its value block to `$svc` — this single template generates all 5 Deployments.
`{{- if $svc.enabled }}`	Skips generating a Deployment entirely for any service flagged disabled.
`{{ $.Values.global.namespace }}`	The `$` prefix re-anchors to the root context — needed because inside a `range`, the dot (`.`) is rebound to the loop item, so `.Values` alone would fail; `$.Values` reaches back to the chart root.
`image: "{{ registry }}/{{ $svc.image.name }}:{{ tag }}"`	Assembles the full image reference from three separately-overridable parts (registry, per-service image name, tag).
`resources: {{ toYaml $svc.resources | nindent 12 }}`	`toYaml` converts the nested resources map back into a YAML block; `nindent 12` re-indents it by 12 spaces so it lines up correctly under the parent key — a standard Helm idiom for embedding structured values.
`--- after {{- end }}{{- end }}`	YAML document separator between each generated Deployment — necessary because the loop emits multiple full documents into one file.

templates/service.yaml & templates/hpa.yaml

Same range-over-services pattern. `hpa.yaml` additionally guards with `{{- if and $svc.enabled $svc.hpa }}` — the extra `$svc.hpa` check means an HPA is only generated for services that define an `hpa:` block in values.yaml (so frontend and admin-service correctly get no HPA object at all, rather than one with empty/zero values).

6. Terraform — AWS Infrastructure (terraform/)

main.tf

BLOCK	PURPOSE
`terraform.required_version`	Pins minimum Terraform CLI version (>=1.6) to avoid syntax/behavior mismatches.
`required_providers.aws ~> 5.0`	Pins the AWS provider to the 5.x major line — allows minor/patch updates but blocks a breaking 6.x upgrade from silently changing behavior.

`backend "s3"`	Remote state storage instead of a local `.tfstate` file — `bucket` / `key` identify where the state JSON lives in S3; `dynamodb_table` provides state locking so two people/processes can't run `apply` concurrently and corrupt state.
`provider "aws".default_tags`	Automatically tags every resource this provider creates with Project/Environment/ManagedBy — avoids repeating tags on every single resource block.

> provider.tf exists but is an empty file — the actual provider configuration lives in main.tf instead. Leftover/unused file from the initial scaffold.

variables.tf

VARIABLE	PURPOSE
`cluster_version = "1.33"`	Bumped up from an original 1.29 default after AWS stopped supporting 1.29 for new cluster creation (Issue #12).
`vpc_cidr / private_subnets / public_subnets`	10.0.0.0/16 VPC split into 2 private (/24) subnets for EKS nodes and 2 public (/24) subnets for the NAT Gateway, ALB, and Jenkins EC2 — spread across two AZs (a/b) for availability.
`node_instance = "t3.small"`	Changed from an original t3.medium default after hitting the account's Free Tier instance-type restriction (Issue #11).
`node_min / node_max / node_desired`	EKS managed node group autoscaling bounds (2 min, 6 max, 2 desired at creation).
`admin_cidr`	The single IP allowed SSH/Jenkins-UI access to the Jenkins EC2 instance — updated repeatedly through the project as the developer's home/network IP changed.

vpc.tf

ARGUMENT	PURPOSE
`source: terraform-aws-modules/vpc/aws`	Uses the official community VPC module instead of hand-writing every subnet/route-table/NAT resource individually.
`enable_nat_gateway + single_nat_gateway = true`	Private subnets need a NAT Gateway for outbound internet (image pulls, package installs); `single_nat_gateway` is an explicit cost optimization — one shared NAT instead of one per AZ (trade-off: less HA, much cheaper, acceptable for a non-production-critical / staging-grade setup).
`public_subnet_tags: kubernetes.io/role/elb = 1`	Required discovery tag so the AWS Load Balancer Controller knows which subnets it's allowed to place an internet-facing ALB into.
`private_subnet_tags: kubernetes.io/role/internal-elb = 1`	Same discovery mechanism for internal-only load balancers placed in private subnets.

eks.tf

ARGUMENT	PURPOSE
`source: terraform-aws-modules/eks/aws ~> 20.0`	Official EKS module — provisions the control plane, IAM roles, OIDC provider, KMS encryption key, and security groups automatically.
`subnet_ids = module.vpc.private_subnets`	Worker nodes live in private subnets (no direct inbound internet exposure); only the control plane endpoint and load balancers are public.
`cluster_endpoint_public_access = true`	Allows kubectl/CI access to the API server from outside the VPC (needed since Jenkins/the developer's laptop aren't inside the VPC) — trade-off is a public API endpoint, mitigated by IAM auth.
`eks_managed_node_groups.default`	One managed node group named "streamingapp-ng" — AWS handles node provisioning, joining, and lifecycle automatically as part of this block.
`capacity_type = "SPOT"`	Cost optimization — Spot instances instead of On-Demand for worker nodes, at the cost of possible interruption (acceptable given the HPA/multi-replica setup can tolerate individual node loss).
`cluster_addons: coredns / kube-proxy / vpc-cni`	Managed EKS add-ons (DNS, network proxy, pod networking) installed and version-managed by AWS instead of manually; `most_recent = true` auto-tracks the latest compatible version.

ecr.tf

ARGUMENT	PURPOSE
`locals.services (list) + for_each = toset(...)`	Single resource block generates all 5 ECR repositories (one per service) by iterating a list, instead of duplicating the resource 5 times.
`image_tag_mutability = "MUTABLE"`	Allows re-pushing to the same tag (needed since the pipeline pushes both a commit-SHA tag and reuses the "latest" tag).
`image_scanning_configuration.scan_on_push`	Automatic vulnerability scanning of every pushed image.
`aws_ecr_lifecycle_policy "keep_last_10"`	Automatically expires older images once a repository has more than 10, keeping storage costs bounded.

s3.tf

RESOURCE	PURPOSE
`aws_s3_bucket.videos`	The bucket storing uploaded video content (`streamingapp-videos`).
`aws_s3_bucket_versioning`	Enabled — protects against accidental overwrite/delete of video files (this same versioning feature was the reason bucket deletion needed the extra version-cleanup step in Issue #32).
`aws_s3_bucket_server_side_encryption_configuration (AES256)`	Encrypts all objects at rest by default without requiring the uploading client to specify encryption headers.

`aws_s3_bucket_public_access_block (all 4 flags true)`	Belt-and-suspenders block against any accidental public exposure of video content — access is only via the app's own signed logic, never directly public.
`aws_s3_bucket_intelligent_tiering_configuration`	Cost optimization — objects untouched for 90+ days automatically move to a cheaper archive access tier without any application-level logic needed.

iam.tf

RESOURCE	PURPOSE
`aws_iam_policy.s3_access`	Custom policy granting `GetObject` / `PutObject` / `ListBucket` scoped only to the videos bucket and its contents (least-privilege — not full S3 access).
`aws_iam_role_policy_attachment.node_s3_access`	Attaches that policy to the EKS node group's IAM role, so pods running on those nodes can reach S3 via the node's instance credentials (a simpler, though less isolated, alternative to per-pod IRSA for this bucket).

security.tf

ARGUMENT	PURPOSE
`aws_security_group.jenkins`	Firewall rules for the Jenkins EC2 instance specifically (separate from the EKS-module-managed cluster/node security groups).
`ingress: port 22 (SSH), cidr_blocks = [var.admin_cidr]`	SSH restricted to a single admin IP, not `0.0.0.0/0` — avoids exposing SSH to the entire internet.
`ingress: port 8080 (Jenkins UI), cidr_blocks = [var.admin_cidr]`	Same restriction for the Jenkins web console.
`egress: all ports/protocols to 0.0.0.0/0`	Outbound left unrestricted — the instance needs to reach package repos, Docker Hub, ECR, GitHub, etc.

ec2.tf

RESOURCE/ARGUMENT	PURPOSE
`data.aws_ami.ubuntu`	Dynamically looks up the latest Ubuntu 22.04 (Jammy) AMI from Canonical's official account (099720109477) rather than hardcoding an AMI ID that would go stale.
`aws_key_pair.streamingapp`	Registers the local public key with AWS so the Jenkins instance can be launched with SSH access pre-configured. Had to be recreated with `-replace` after a stray manually-created key pair under the same name caused a real key mismatch (Issue #14).
`aws_instance.jenkins`	The Jenkins CI server — t3.small, placed in the first public subnet, with a public IP so Ansible/the developer can reach it directly over SSH/HTTP.
`tags: Role = "jenkins"`	Consumed directly by Ansible's dynamic inventory (`tag_Role_jenkins` group) — this tag is what makes the EC2 instance auto-discoverable as an Ansible target.

outputs.tf

Exposes key values after `apply` for use by later pipeline stages / manual commands — cluster endpoint and name (for `kubectl` / `aws eks update-kubeconfig`), ECR repo URLs (for image push targets), S3 bucket name, and the Jenkins instance's public IP (for SSH/Ansible).

7. Ansible — Server Configuration (ansible/)

inventory/aws_ec2.yml

FIELD	PURPOSE
`plugin: aws_ec2`	Dynamic inventory plugin — queries live AWS EC2 state instead of a static list of IPs, so newly created/destroyed instances are picked up automatically.
`filters: tag:Project=StreamingApp, instance-state-name=running`	Only considers this project's instances, and only ones actually running (skips terminated/stopped).
`keyed_groups: key=tags.Role, prefix=tag_Role`	Auto-creates Ansible groups from the EC2 `Role` tag (e.g. an instance tagged `Role=jenkins` lands in group `tag_Role_jenkins`) — this is exactly the group `site.yml` targets.
`hostnames: private-ip-address`	Uses the instance's private IP as its inventory hostname/identity.
`compose.ansible_host: public_ip_address`	Overrides the actual connection address to the public IP, since Ansible runs from outside the VPC.
`compose.ansible_user / ansible_ssh_private_key_file`	Sets the SSH login user (`ubuntu`, standard for Ubuntu AMIs) and private key path — quoted with nested single-quotes because `compose` values are evaluated as Jinja2 expressions, not literal strings.

site.yml

Top-level playbook — two plays, each targeting a different inventory group and role set.

FIELD	PURPOSE
`hosts: tag_Role_jenkins`	First play targets only the Jenkins instance; applies common, docker, kubectl, then jenkins roles in that order (jenkins last since it depends on Docker/kubectl being present).
`hosts: tag_Role_appnode`	Second play targets any instance tagged as an app node (not currently used in the AWS deployment since app workloads run on EKS, not raw EC2 — kept for flexibility/local testing scenarios).
`become: true`	Runs all tasks with sudo — needed since package installation, systemd management, etc. require root.

roles/common/tasks/main.yml

TASK	PURPOSE
`apt: update_cache + upgrade=dist`	Full system update before anything else installs, so subsequent package installs pull from current repo metadata.
`Install & enable NTP`	Accurate clock sync — important for TLS certificate validation and any time-sensitive operations (e.g. AWS request signing).
`swapoff -a + remove swap from fstab`	Kubernetes/kubelet requires swap disabled on nodes; applied here as a general hygiene step even though these instances aren't EKS worker nodes.

roles/common/handlers/main.yml

Defines the `restart ntp` handler, triggered by the "Ensure NTP is enabled" task via `notify` — handlers only run once at the end of a play even if notified multiple times, and only if the notifying task actually reported a change.

roles/docker/tasks/main.yml

TASK	PURPOSE
`Install prerequisites (apt-transport-https, ca-certificates, curl, gnupg, lsb-release)`	Needed to add and trust a third-party apt repository over HTTPS.
`apt_key (Docker GPG)`	Registers Docker's signing key so apt trusts packages from their repo.
`apt_repository with {{ ansible_lsb.codename }}`	Dynamically inserts the detected Ubuntu codename (e.g. "jammy") so the same task works across Ubuntu versions without hardcoding.
`Install docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin`	Full Docker Engine + CLI + container runtime + BuildKit-based build plugin.
`user: name=jenkins, groups=docker, append=yes`	Lets the Jenkins service user run `docker` commands without sudo — `append: yes` is critical, without it this would replace all of the jenkins user's other group memberships instead of adding to them.

roles/jenkins/tasks/main.yml

TASK	PURPOSE
`Install openjdk-21-jre`	Current Jenkins LTS requires Java 21 at minimum — installing Java 17 here was the direct cause of Jenkins failing to start (Issue #19).
`alternatives: name=java, path=.../java-21-openjdk-amd64/bin/java`	Explicitly sets Java 21 as the system default `java` binary — needed because multiple JDKs can coexist and the "current" one is controlled by the alternatives system, not just install order.
`file: /etc/apt/keyrings, state=directory`	Ensures the modern keyring directory exists before downloading a key into it.
`get_url (jenkins.io-2026.key) → /etc/apt/keyrings/jenkins-keyring.asc`	Modern signed-by approach replacing the deprecated `apt_key` module — the "2026" in the URL reflects Jenkins' own key rotation, discovered mid-project when the original key URL had expired.
`apt_repository with signed-by=/etc/apt/keyrings/jenkins-keyring.asc`	Explicitly ties this one repository to this one key, rather than trusting it globally like the old apt-key approach did.
`Install & enable jenkins service`	Installs the Jenkins package and starts/enables the systemd service.

roles/kubectl/tasks/main.yml

TASK	PURPOSE
`get_url: kubectl v1.29.0 → /usr/local/bin/kubectl, mode 0755`	Downloads a specific pinned kubectl binary directly (not via apt) and makes it executable.
`shell: get-helm-3 script, args.creates: /usr/local/bin/helm`	Runs the official Helm install script; `creates:` makes the task idempotent — Ansible skips re-running it if the target file already exists.
`apt: unzip`	Added specifically to fix Issue #17 — the AWS CLI installer needs `unzip`, which isn't present by default.
`shell: download+unzip+install AWS CLI v2, args.creates: /usr/local/bin/aws`	Same idempotency pattern as Helm — three shell steps chained because the official AWS CLI installer is a self-extracting installer script, not a single binary.
`unarchive: Terraform 1.7.0 zip, remote_src=yes → /usr/local/bin/`	Downloads and extracts directly on the remote host (`remote_src: yes` avoids

	downloading to the Ansible control machine first and copying over).

8. Jenkinsfile — CI/CD Pipeline

BLOCK/DIRECTIVE	PURPOSE
`agent any`	Runs on any available Jenkins agent/executor (single-node setup here — just the one EC2 instance).
`environment: AWS_ACCOUNT_ID = credentials('aws-account-id')`	Pulls a Jenkins credential store secret into an env var — never hardcoded in the file itself.
`IMAGE_TAG = "${GIT_COMMIT[0..7]}"`	Short 8-character commit SHA used to tag every image — makes each build's images traceable back to an exact commit and guarantees a unique tag per build (unlike relying only on "latest").
`options.buildDiscarder(logRotator(numToKeepStr:'10'))`	Keeps only the last 10 build logs/artifacts, preventing unbounded disk growth on the Jenkins server.
`options.timeout(45 MINUTES)`	Hard ceiling on total pipeline runtime — kills a hung build automatically.
`options.disableConcurrentBuilds()`	Prevents two builds of the same job running simultaneously — important since a concurrent Terraform apply against the same state would conflict.
`stage('Checkout')`	`checkout scm` pulls the exact commit that triggered the build; `git log --oneline -5` is a diagnostic print, not functionally required.
`stage('Unit Tests') → parallel{}`	Runs the three test sub-stages (auth, streaming, frontend) concurrently rather than sequentially, shortening total pipeline time.
`npm test -- --watchAll=false`	`--watchAll=false` is required for CRA-based test runners so Jest runs once and exits instead of entering interactive watch mode, which would hang the pipeline forever.
`--passWithNoTests (frontend only)`	Prevents a hard failure if the frontend currently has zero test files, while still leaving the door open for real tests later.
`stage('Docker Build & Push') → services list`	Defines context/dockerfile per service explicitly (not a single shared convention) —

	corrects Issue #27, where a uniform assumption broke several services' builds.
`aws ecr get-login-password | docker login --password-stdin`	Authenticates the Docker CLI against ECR using a short-lived token rather than long-lived static credentials.
`docker build -f ... -t ... / docker push / tag+push :latest`	Each service is built once, pushed under its unique commit-SHA tag, then re-tagged and pushed again as `:latest` for anything that always wants the newest image.
`stage('Terraform Plan') → when { branch 'main' }`	Infra changes are only even previewed on the main branch — feature branches never touch Terraform state at all.
`withCredentials([[$class:'AmazonWebServicesCredentialsBinding', ...]])`	Injects AWS credentials into the shell step's environment only for its duration, scoped and short-lived rather than exported globally in the pipeline.
`terraform init -backend-config=...`	Backend config values passed on the CLI instead of hardcoded in main.tf's backend block — allows the same code to target different state locations without editing source (though in this project a single fixed backend was used throughout).
`stage('Terraform Apply') → input { message ... }`	Manual approval gate — a human must click "proceed" in the Jenkins UI before any real infrastructure change is applied, even on main.
`stage('Ansible Configure')`	Runs the same playbook covered in Section 7, with `--extra-vars "env=prod"` passed through for any environment-conditional logic in the roles.
`stage('Deploy to EKS')`	`aws eks update-kubeconfig` points kubectl/helm at the right cluster; `helm upgrade --install ... --wait --timeout 5m` deploys and blocks until all resources report ready (or fails the build after 5 minutes if they don't).
`--set global.imageTag=${IMAGE_TAG}`	Overrides the placeholder from values-prod.yaml with this build's actual commit-SHA tag — ties every deploy to a specific, traceable image build.

`stage('Smoke Tests')`	Post-deploy validation — spins up a throwaway curl pod per service (`kubectl run ... --rm -i --restart=Never`) hitting each service's ClusterIP DNS name directly (bypassing the ALB) to confirm the deployment is actually healthy from inside the cluster, not just "rolled out."
`post.success / post.failure → slackSend`	Notifies a Slack channel on build outcome, including the build number, branch, and (on success) the deployed image tag.
`post.always → cleanWs()`	Wipes the Jenkins workspace after every build regardless of outcome, so builds start from a clean checkout each time and disk usage doesn't accumulate.

9. Command Reference by Phase

IAM & AWS CLI Setup

```
aws configure                    # set access key / secret / region for the IAM user
aws sts get-caller-identity      # confirm which identity the CLI is authenticated as
```

Terraform — Infra Provisioning

```
terraform init -backend-config="bucket=streamingapp-tfstate" \
               -backend-config="key=eks/terraform.tfstate" \
               -backend-config="region=us-east-1"
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
terraform apply -replace="aws_key_pair.streamingapp" -replace="aws_instance.jenkins"   # forced recreation
(Issue #14)
terraform state list               # inspect what's under management
terraform output
```

Ansible — Jenkins Server Configuration

```
ansible-playbook -i inventory/aws_ec2.yml site.yml --extra-vars "env=prod"
ansible-inventory -i inventory/aws_ec2.yml --graph     # verify dynamic inventory is resolving instances
```

Docker Images — Build & Push to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
<account>.dkr.ecr.us-east-1.amazonaws.com
docker build -f frontend/Dockerfile -t <registry>/streamingapp/frontend:<tag> frontend
docker push <registry>/streamingapp/frontend:<tag>
docker tag <registry>/streamingapp/frontend:<tag> <registry>/streamingapp/frontend:latest
docker push <registry>/streamingapp/frontend:latest
# repeated per service (auth-service, streaming-service, admin-service, chat-service)
```

Application Deployment — Helm to EKS

```
aws eks update-kubeconfig --name streamingapp-cluster --region us-east-1
helm lint ./helm/streamingapp
helm template ./helm/streamingapp -f helm/values-prod.yaml   # render-only sanity check
helm upgrade --install streamingapp ./helm/streamingapp \
  --namespace streamingapp --create-namespace \
  -f helm/values-prod.yaml \
  --set global.imageRegistry=<account>.dkr.ecr.us-east-1.amazonaws.com \
  --set global.imageTag=<commit-sha> \
  --wait --timeout 5m
kubectl get pods -n streamingapp
kubectl get svc -n streamingapp
kubectl describe pod <pod> -n streamingapp     # diagnosing CrashLoopBackOff / ImagePullBackOff
```

Persistent Storage — EBS CSI Driver

```
aws eks create-addon --cluster-name streamingapp-cluster --addon-name aws-ebs-csi-driver \
  --service-account-role-arn <IRSA-role-arn>
kubectl apply -f gp3-storageclass.yaml
kubectl delete pvc mongo-data-mongo-0 -n streamingapp
kubectl delete pod mongo-0 -n streamingapp     # forces PVC/pod recreation against the new StorageClass
```

Networking — ALB Ingress

```
kubectl apply -f k8s/ingress-frontend.yaml
kubectl apply -f k8s/ingress-api.yaml
kubectl delete ingress streamingapp-ingress -n streamingapp   # remove the old single-Ingress object
kubectl get ingress -n streamingapp
aws elbv2 describe-load-balancers --query "LoadBalancers[].DNSName"
aws elbv2 describe-target-health --target-group-arn <arn>      # confirm targets healthy
```

Jenkins Pipeline Testing

```
sudo -u jenkins /usr/bin/jenkins        # manual foreground run to surface the real startup error (Issue #19)
sudo systemctl status jenkins
sudo systemctl restart jenkins
sudo journalctl -u jenkins -f           # tail live Jenkins service logs
```

Verification

```
curl -I http://<alb-dns-name>/                # frontend reachable
curl http://<alb-dns-name>/api/auth/health     # API routes reachable through the ALB
kubectl get hpa -n streamingapp                # confirm autoscalers are reading metrics
```

Teardown / Cleanup

```
# preferred path (not available once state was lost — Issue #31):
# terraform destroy

# manual dependency-ordered deletion actually used:
aws eks delete-nodegroup --cluster-name streamingapp-cluster --nodegroup-name streamingapp-ng
aws eks delete-cluster --name streamingapp-cluster
aws ec2 terminate-instances --instance-ids <jenkins-instance-id>
aws ec2 delete-nat-gateway --nat-gateway-id <id>
aws ec2 release-address --allocation-id <eip-id>
aws ecr delete-repository --repository-name streamingapp/<service> --force
aws s3api list-object-versions --bucket <bucket>   # then delete-object per version+delete-marker (Issue #32)
aws s3 rb s3://<bucket> --force
aws kms delete-alias --alias-name alias/eks/streamingapp-cluster
aws kms schedule-key-deletion --key-id <id> --pending-window-in-days 7
aws iam detach-role-policy --role-name <role> --policy-arn <arn>
aws iam delete-role --role-name <role>
aws iam delete-policy --policy-arn <arn>
aws ec2 describe-security-group-rules --filters "Name=group-id,Values=<sg-id>"
aws ec2 revoke-security-group-ingress --group-id <sg-id> --security-group-rule-ids <rule-id>   # (Issue #33)
aws ec2 delete-security-group --group-id <sg-id>
aws ec2 delete-subnet --subnet-id <id>
aws ec2 delete-route-table --route-table-id <id>
aws ec2 detach-internet-gateway / delete-internet-gateway
aws ec2 delete-vpc --vpc-id <id>
aws ec2 delete-key-pair --key-name streamingapp-key   # (Issue #34)
```

End of technical reference — covers every Docker, Compose, Kubernetes, Helm, Terraform, Ansible, and Jenkins file created for this project, plus the commands executed at each phase.